

ATC Readback Verifier

NLP Group Project | Option 1: Application Development

Project topic proposal for approval

Overview

We propose to build an **ATC Readback Verifier**, an NLP application that checks whether a pilot's readback correctly matches an air traffic control instruction and flags any discrepancy.

The problem

In aviation, a controller issues an instruction and the pilot reads it back so the controller can confirm it was heard correctly. Incorrect or incomplete readbacks, such as a wrong altitude, transposed digits, or an omitted item, are a well documented contributor to safety incidents and are recorded in public databases such as NASA's Aviation Safety Reporting System. Existing tools focus on transcribing ATC audio rather than verifying that a readback is actually correct. Our system targets that gap.

What we will build (proof of concept)

A simple web application where the user enters the controller instruction and the pilot readback as text. The system extracts the key items from each message (callsign, altitude, heading, speed, frequency, squawk code, runway) and compares them field by field, returning a clear verdict: a match, or a list of the specific discrepancies found. Scope note: the proof of concept works on text input. Converting live radio audio to text is a planned future direction and is not part of the demonstrated system.

Evaluation

We will build our own test set of roughly 40 to 60 instruction and readback pairs, each labelled with the correct outcome and deliberately covering both correct readbacks and every error type. We will report precision, recall, and F1 for error detection, plus the false alarm rate on correct readbacks. We will also document 5 to 10 failure cases with hypotheses about why they occur.

Why this topic fits the brief

- **Relevance:** a real safety problem in a high stakes domain, with active research on ATC language.
- **Feasibility:** a clear, modest POC that the team can complete in the time available.
- **Originality:** not a standard chatbot or summariser; the comparison logic and the test set are our own work.

Tools

Python with a Streamlit interface for the application, an LLM API for the extraction step, and a GitHub repository with a README and requirements file for reproducibility.